15 Exploring Optimization and Scaling of LLMs

Training Large Language Models (LLMs) is not only about achieving high model performance—as you may have already inferred at this point in the book or course—but also about doing so efficiently.

This chapter presents a real-world case study to introduce practical optimization techniques and subsequent parallelization strategies that improve both the efficiency and scalability of LLM training. Specifically, we focus on hands-on results obtained by training the LLaMA 3.2–1B model on the ACC partition of the MareNostrum 5 supercomputer. The tasks proposed in this chapter also requires use the facebook/opt-1.3b model. Both models offers a similar parameter scale while providing a stable and well-documented baseline on Hugging Face, making them ideal for reproducible educational experiments.

In addition, we use this chapter to demonstrate how Hugging Face's `Trainer` API—introduced in Chapter 13—can be leveraged to simplify the distributed training process. As shown previously, this high-level abstraction greatly reduces the need for the low-level manual configuration required when using native PyTorch, as we explored in Chapters 11 and 12.

It is worth noting that this duality in expressing parallelism throughout the book is intentional. By covering both approaches—from granular control with PyTorch DDP to the abstraction provided by `Trainer`—we aim to give readers a well-rounded understanding of the training software stack. This foundation empowers students to explore further on their own and to choose

the right tool for the task at hand—whether they need low-level flexibility or high-level productivity.

For this reason, the current chapter does not delve deeply into the analysis of training results, as we did in Chapters 11 and 12. We leave this to the reader as an optional exercise, following the same methodology presented in those earlier chapters. Instead, we focus on demonstrating how to first optimize and then scale LLM training, putting into practice the layered architecture and execution flow for distributed training with Hugging Face `Trainer`, as illustrated in Figure 13.5.

15.1 Case study and code

LLaMA 3.2 with 1B parameters

This section introduces the practical foundation for the optimization and scaling experiments discussed throughout the chapter. We will work with two real-world large language models: the open-source LLaMA 3.2 1B model (`Llama-3.2-1B`) with 1 billion parameters, and OPT-1.3B model (facebook/opt-1.3b), a 1.3B-parameter model from the OPT family.

While both models are considered small by today's LLM standards, they preserve the core architectural and computational characteristics of much larger transformers, making them especially well suited for controlled benchmarking, optimization studies, and scaling experiments on MareNostrum 5. Throughout this chapter, we will present and analyze the experimental results obtained with LLaMA 3.2 (1B), while students will be asked to reproduce the same experiments using OPT-1.3B and compare both behaviors.

Both models were downloaded from the Hugging Face Model Hub and transferred to the GPFS filesystem of MareNostrum 5, following the same procedure introduced in the previous chapter.

Synthetic Data

To emphasize computational performance rather than model accuracy, we use a synthetic dataset of 10,000 samples generated dynamically. This approach removes variability introduced by I/O operations or preprocessing, ensuring that performance metrics reflect the computational efficiency of the training process. This method is common in benchmarking studies, allowing us to focus on GPU utilization, training throughput, and memory usage without interference from data loading bottlenecks.

Baseline Training Script

We establish a baseline using the Python script `benchmark.py`, which trains the LLaMA 3.2 model on a single GPU using Hugging Face's Trainer API. This script collects essential performance metrics, serving as a reference point for evaluating the impact of optimizations explored in later sections.

In summary, the `benchmark.py` script:

- Defines model precision (either bf16 or fp32) to evaluate memory efficiency and numerical stability.
- Loads a pretrained model (AutoModelForCausalLM) from Hugging Face.
- Generates a synthetic dataset (DummyDataset).
- Initializes the Trainer API with appropriate training arguments.
- Logs key performance metrics, including Training Throughput and Max Reserved GPU Memory.

The code listing is as follows:

```
import os
import torch

from transformers import AutoModelForCausalLM, Trainer
from utils import DummyDataset, get_args, log_rank, report_memory

def main(training_arguments, benchmark_arguments):
    # Set the data type for model parameters (bf16 or fp32)
    TORCH_DTYPE = torch.bfloat16 if benchmark_arguments.model_precision == "bf16"
```

```
                                    else torch.float
    world_size = int(os.environ["WORLD_SIZE"])


    model = AutoModelForCausalLM.from_pretrained(
        benchmark_arguments.path_to_model,
        attn_implementation=benchmark_arguments.attn,
        torch_dtype=TORCH_DTYPE
    )

    train_dataset = DummyDataset(
        benchmark_arguments.sequence_length,
        benchmark_arguments.num_samples
    )

    trainer = Trainer(
        model=model,
        args=training_arguments,
        train_dataset=train_dataset
    )

    metrics = trainer.train().metrics
            log_rank(f"Training throughput:    {training_arguments.max_steps *
            training_arguments.per_device_train_batch_size *
            benchmark_arguments.sequence_length/metrics['train_runtime']:.2f}
            Tokens/s/GPU")

    report_memory()

if __name__ == "__main__":
    _training_arguments, _benchmark_arguments = get_args()
    main(_training_arguments, _benchmark_arguments)
```

The `utils.py` module, used in conjunction with this script, provides utility functions and classes and should reside in the same directory. No modifications are required.

Performance Metrics: Measuring Training Efficiency

This script collects two key metrics:

- Training Throughput (Tokens/s/GPU): Quantifies the speed at which tokens are processed. It is calculated as:

$$\text{Training Throughput} = \frac{\text{max\_steps} \times \text{per\_device\_train\_batch\_size} \times \text{sequence\_length}}{\text{train\_runtime}}$$

- Max Reserved GPU Memory (GB): Obtained using `torch.cuda.max_memory_reserved()`, this value indicates peak memory usage during training.

These metrics allow us to evaluate how different configurations affect efficiency and scalability.

Example output from a baseline run:

```
INFO -> Training throughput:     27542.25 Tokens/s/GPU
INFO -> Max reserved GPU Memory: 61.59
```

SLURM Job Submission Script

As in Chapters 11 and 12, we use `torchrun` in a SLURM environment. The Python code now relies on the Hugging Face Trainer API, but the job launcher strategy remains familiar. The SLURM script is available in the GitHub repository accompanying this book.

To explore performance systematically, only a few variables need to be modified in the SLURM submission script. The following variables are fixed to ensure consistency across experiments:

- `MAX_STEPS`: Maximum number of training steps.
- `SEQUENCE_LEN`: Input sequence length in tokens. (Kept short to accommodate course resource limitations.)
- `OPTIMIZER`: Optimizer used; fixed to AdamW.
- `TORCH_COMPILE`: Whether to use `torch.compile`. Disabled due to current performance issues with Liger kernels.

The variables to be changed across experiments include:

- `MICRO_BATCH_SIZE`: Number of samples per device in each forward/backward pass.
- `MODEL_PRECISION`: Precision format (fp32, bf16).
- `MIXED_PRECISION`: Enables AMP (Automatic Mixed Precision).

- `ATTN`: Attention implementation: eager, sdpa, flash_attention_2.
- `LIGER_KERNEL`: Whether optimized kernels are used.

These will be introduced in detail in each subsequent experiment.

Running the Baseline Experiment

We begin by running the experiment with the default SLURM script (available in the GitHub repository). Results are recorded in a Table 15.1 that tracks the value of each variable per run.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	**Training Throughput**	**Memory (GB)**
15.1	1	6	fp32	No	eager	false	**10485**	**61,89**

Table 15.1 – Baseline training results using LLaMA 3.2-1B model.

Task 15.1 – Baseline Experiments

Replicate the baseline training experiment for the LLaMA 3.2 1B (`Llama-3.2-1B`) and OPT-1.3B (facebook/opt-1.3b), which were downloaded in the previous chapter, using the default hyperparameters defined in the SLURM file according to Table 15.1.

For this initial baseline, it is recommended to start with a batch size of 6 for LLaMA 3.2 (1B) and a batch size of 4 for OPT-1.3B, as a safe initial configuration to ensure stable execution.

All the exercises in this chapter are designed to be carried out exclusively by modifying the SLURM job scripts. The SLURM template can be obtained from GitHub, and no Python files need to be modified.

Proceed as follows:

- First, run the experiment with the LLaMA-based configuration to verify that you obtain results consistent with those reported in this chapter.
- Then, repeat exactly the same procedure using the OPT-based configuration.

The results obtained in this task should be carefully recorded, as they will establish the baseline performance against which all subsequent optimization and scaling experiments in this chapter will be compared.

15.2 Efficient Training on a single GPU

Before scaling across multiple GPUs or nodes, it is crucial to identify the optimal training configuration on a single GPU. As we discussed in Chapter 10, training throughput is heavily influenced by hyperparameters such as batch size, numerical precision, and the efficiency of key kernel implementations. In this section, we conduct a series of controlled experiments to progressively optimize model training performance on a single NVIDIA H100 GPU. The results guide us in understanding how each factor contributes to memory usage, training speed, and overall GPU utilization.

We begin by exploring batch size scaling limits under full precision, and then investigate how techniques like mixed precision, model precision tuning, Flash Attention, and Triton-based Liger kernels impact training efficiency.

Batch Size Scaling

One of the most effective ways to improve training efficiency is to increase the batch size to better utilize the GPU's available memory. In our SLURM scripts, the batch size is controlled using the `MICRO_BATCH_SIZE` variable, which determines the number of samples processed per GPU in a single forward and backward pass.

We begin with a conservative batch size and incrementally increase it until the GPU runs out of memory (OOM). This trial-and-error process reveals the memory limit and serves as a baseline for further optimizations.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	**Training Throughput**	**Memory (GB)**
15.1	1	6	fp32	No	eager	false	**10,485**	**61,89**
15.2	1	7	fp32	No	eager	false	**-**	**OOM**

Table 15.2 – Training throughput and memory usage using fp32 precision.

Task 15.2 – Finding the Out-of-Memory (OOM) Limit
Using the SLURM script from github used for Task 15.1 as a starting point, replicate the baseline experiment and progressively increase the `MICRO_BATCH_SIZE` until an Out-of-Memory (OOM) condition is reached.
Proceed in two stages:

- First, perform this experiment with LLaMA 3.2 (1B) to verify that you correctly understand the methodology and can reliably identify the OOM point.
- Then, repeat exactly the same procedure with OPT-1.3B.

When the OOM occurs, capture the `torch.cuda.OutOfMemoryError` message (from standard error file) and record the requested memory allocation size that could not be satisfied. From this experiment, determine the maximum batch size that fits in GPU memory under fp32 precision for each model.
Finally, compare and discuss both results. What differences do you observe between LLaMA 3.2 (1B) and OPT-1.3B? How does the increase from 1B to 1.3B parameters reflect on the attainable batch size and memory pressure?

Enhancing Training Efficiency with Mixed Precision

Mixed precision training is a well-established technique (Chapter 11) that combines lower and higher numerical precisions to accelerate training[52] and reduce memory usage. In our case, we use bfloat16 (`bf16`) for its wider dynamic range compared to fp16, which improves numerical stability without requiring loss scaling. On H100 GPUs, bf16 delivers significant speedups while maintaining precision, making it an ideal candidate for LLM training.

To activate mixed precision, we modify the `MIXED_PRECISION` variable in the SLURM script. In this experiment, switching from fp32 to bf16 mixed precision allows us to train with a batch size of 5—just below the OOM threshold—and observe modest memory savings and performance gains.

[52] Training precision refers to how data and gradients are stored during training; model precision refers to how the weights are stored during inference.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	**Training Throughput**	**Memory (GB)**
15.3	1	5	fp32	bf16	eager	false	**11.489**	**58,27**

Table 15.3 – Training with mixed precision (bf16) reduces memory consumption and increases throughput.

Although mixed precision is often associated with reduced memory usage and the ability to increase batch size, this expectation does not always hold in practice, especially when training large language models close to the Out-Of-Memory boundary. When switching from fp32 to bf16 on modern accelerators such as the NVIDIA H100, the memory footprint of activations and gradients is indeed reduced, and computational throughput improves thanks to specialized Tensor Core units. However, these components represent only a fraction of the total GPU memory consumption during training.

A dominant portion of memory is instead occupied by the optimizer states, particularly when using adaptive optimizers such as AdamW. For each model parameter, AdamW maintains additional auxiliary tensors to store first and second moments, which are typically kept in fp32 for numerical stability, regardless of whether the model itself is trained in bf16. As a consequence, even when the model weights and gradients are stored in reduced precision, a large fraction of the total memory footprint remains unaffected by mixed precision.

In addition to optimizer states, GPU memory is also consumed by temporary CUDA buffers, PyTorch runtime allocations, communication workspaces, and internal tensor conversions required by certain kernels. On H100-class GPUs, enabling bf16 activates different execution paths and kernel implementations that may introduce additional transient buffers. When training is already operating near the memory capacity of the device, these extra allocations—although small in relative terms—can become decisive and trigger an Out-Of-Memory condition earlier than expected.

Another important factor is memory fragmentation. The dynamic nature of tensor allocation and deallocation during forward and backward passes

can lead to non-contiguous free memory regions. Even when the total free memory appears sufficient, fragmentation can prevent the allocation of a large contiguous block required for a new batch, again leading to an OOM error. This effect becomes more pronounced when working at the edge of GPU capacity, where even slight changes in allocation patterns can have visible consequences.

For all these reasons, it is entirely possible—and indeed common in practice—that activating bf16 mixed precision improves throughput while leaving the maximum achievable batch size unchanged, or even slightly reduced, when compared to fp32. Mixed precision should therefore be understood primarily as a performance optimization technique, not as a guaranteed method for increasing batch size. Its real benefits emerge from faster computation, better hardware utilization, and reduced pressure on specific parts of the memory hierarchy, rather than from a uniform reduction of the total memory footprint.

Task 15.3 – Mixed Precision Training

Starting from the configuration used in Task 15.2, modify the SLURM script to enable `bf16` mixed precision by activating the `MIXED_PRECISION` variable. Use this configuration to evaluate the impact of mixed precision on training throughput and GPU memory consumption.

Proceed again in two stages:

- First, run the experiment with LLaMA 3.2 (1B) using the same hyperparameters reported in this chapter, and verify that your results are consistent with the reference measurements in terms of throughput, memory usage, and stability.
- Then, repeat exactly the same experiment with OPT-1.3B.

As part of this experiment, progressively increase the batch size again under `bf16` precision until you reach the new OOM limit. Note that, despite using mixed precision, it may still be necessary to reduce the batch size with respect to fp32 for stability reasons, depending on the memory pressure introduced by the optimizer and activation checkpoints.

For both models, record data obtained.

Finally, compare and discuss the results obtained with LLaMA 3.2 (1B) and OPT-1.3B. In particular[53]:

- Does mixed precision lead to similar throughput improvements for both models?
- Does the maximum batch size under bf16 decrease in both cases?
- How does the difference in parameter count (1B vs. 1.3B) reflect on memory usage and attainable batch size under mixed precision?

Fine-Tuning Model Precision

Beyond mixed precision, memory usage can be further reduced by switching the entire model to a lower numerical precision. This means storing model weights, activations, and gradients directly in bf16 format, instead of maintaining fp32 master copies as is typically done in mixed precision training. By removing these high-precision replicas, the memory footprint of the training process is significantly reduced.

Setting the `MODEL_PRECISION` variable to `bf16` in the SLURM script activates this mode. In our experiments on the NVIDIA H100, this results in a substantial drop in memory usage—close to 10 GB saved—together with a noticeable increase in training throughput. This optimization alleviates memory pressure and may enable larger batch sizes, although the exact limit remains constrained by optimizer states, activation memory, and runtime buffers, as discussed previously.

In this regime, the model operates entirely in bf16, and performance improvements now stem not only from faster computation on Tensor Cores, but also from the reduced memory traffic associated with lower-precision tensors. However, because numerical stability margins are narrower than in mixed precision, this configuration must be validated carefully, especially for longer training runs or more sensitive optimization settings.

[53] This analysis will allow you to assess whether the benefits and limitations of mixed precision generalize across different transformer architectures, even when model sizes are relatively close.

Task 15.4 – Full Model Precision (bf16)

Train the model using full bf16 precision by setting both `MODEL_PRECISION=bf16` and `MIXED_PRECISION=bf16` in the SLURM script, thereby eliminating the use of fp32 master weights. This configuration forces model weights, activations, and gradients to be stored entirely in bf16 format.

Proceed again in two stages:

- First, run the experiment with LLaMA 3.2 (1B) and measure training throughput and peak GPU memory consumption. Compare your results with those obtained in Table 15.4.
- Then, repeat exactly the same experiment using OPT-1.3B.

Finally, compare and discuss the results for LLaMA and OPT. Does full bf16 model precision provide similar benefits for both models? Are the memory savings and throughput gains proportional to the difference in parameter count?

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	**Training Throughput**	**Memory (GB)**
15.4	1	5	bf16	bf16	eager	false	**13.520**	**47,66**

Table 15.4 – Using bf16 model precision further improves throughput and reduces memory usage.

With memory freed by using bf16 for both model and mixed precision, we can now increase the batch size. Using the same configuration, we gradually scale up the batch size and find that the maximum batch size without an OOM error is now 7—up from 5.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	**Training Throughput**	**Memory (GB)**
15.5	1	7	bf16	bf16	eager	false	**14.315**	**61,32**
15.5	1	8	bf16	bf16	eager	false	**-**	**OOM**

Table 15.5 – Model precision optimizations allow a larger batch size and improved throughput.

Task 15.5 – Increasing Batch Size with Full Model Precision
Using the configuration obtained in Task 15.4, progressively increase the batch size in order to identify the new memory ceiling under full bf16 model precision.
Perform this scaling experiment for both, LLaMA 3.2 (1B) and OPT-1.3B models.
For each model, determine:

- The maximum batch size that fits in memory without triggering an OOM error
- The corresponding training throughput
- The resulting peak GPU memory consumption

Then, compare the behavior of both models and analyze:

- How much the maximum batch size increases with respect to the mixed precision setup
- Whether the increase is similar for LLaMA and OPT
- How the difference in parameter count (1B vs. 1.3B) impacts the attainable batch size under full bf16

This task consolidates the relationship between numerical precision, memory pressure, and scalability, and highlights why full bf16 precision can unlock additional performance headroom—within carefully controlled stability margins.

Enhancing Attention Mechanisms

The self-attention mechanism at the core of Transformer architectures is one of the most computationally and memory-intensive components of large language models. In its standard implementation, the attention operation explicitly materializes large attention matrices whose memory footprint grows quadratically with the sequence length. As a result, self-attention often becomes the dominant bottleneck in both memory consumption and execution time, especially when training with large batch sizes or long sequences.

Flash Attention is a highly optimized attention algorithm designed to alleviate this bottleneck. Instead of explicitly storing full attention matrices in GPU memory, Flash Attention computes the attention operation in small,

register-resident blocks, fusing multiple operations and reducing the number of memory reads and writes. This dramatically lowers memory traffic and improves arithmetic intensity, allowing the GPU to operate much closer to its theoretical peak performance.

By switching the `ATTN` variable to `sdpa` in the SLURM script, we enable PyTorch's native implementation of Flash Attention. This change alone is often sufficient to produce a substantial reduction in memory usage together with a significant increase in training throughput, without modifying any Python code. Flash Attention therefore represents a pure runtime optimization that directly translates into higher efficiency and additional memory headroom.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	**Training Throughput**	**Memory (GB)**
15.6	1	7	bf16	bf16	sdpa	false	**24.446**	**39,27**

Table 15.6 – Flash Attention drastically reduces memory usage and boosts throughput.

Task 15.6 – Enabling Flash Attention

Starting from the configuration obtained in Task 15.5 (full bf16 model precision), update the SLURM script to enable Flash Attention by setting:

```
ATTN="sdpa"
```

Proceed in two stages:

- First, run the experiment with LLaMA 3.2 (1B) and verify that your measured training throughput and memory usage are consistent with the reference results reported in Table 15.6.
- Then, repeat exactly the same experiment using facebook/opt-1.3b.

Compare and discuss the results obtained with LLaMA and OPT. Does Flash Attention provide similar throughput gains and memory savings for both models? Are the benefits proportional to the model size?

The memory savings introduced by Flash Attention can be immediately reinvested in larger batch sizes, higher throughput, or a combination of both. For LlaMA model, we find that the model can now fit a batch size of 14 on a single GPU.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	**Training Throughput**	**Memory (GB)**
15.7	1	14	bf16	bf16	sdpa	false	**27.669**	**60,89**

Table 15.7 – Flash Attention enables a significantly larger batch size and higher throughput.

> **Task 15.7 – Increasing Batch Size with Flash Attention**
> Using the Flash Attention configuration from Task 15.6, progressively increase the batch size until an Out-of-Memory (OOM) error is reached.
> Perform this scaling experiment for both models and compare the results with those obtained without Flash Attention and analyze:
> - How much the maximum batch size increases thanks to Flash Attention
> - Whether this increase is similar for LLaMA and OPT
> - The combined effect of full bf16 precision + Flash Attention on overall scalability

This task demonstrates how algorithmic kernel-level optimizations can unlock a second level of scaling beyond what is achievable through numerical precision alone.

Accelerating Training with Triton and Liger Kernels

As a final optimization step in our single-GPU workflow, we enable Liger Kernels, an open-source collection of highly optimized GPU kernels written in Triton, a language and compiler developed by OpenAI. Triton allows researchers to implement high-performance GPU code using a higher-level programming model than CUDA, while still achieving near-hardware-optimal efficiency.

Liger kernels target memory-bound operations that dominate Transformer training, such as normalization layers, linear projections, and activation functions. By fusing multiple operations and reducing memory traffic, Liger significantly improves both memory efficiency and computational throughput.

Liger kernels are integrated into Hugging Face Transformers, and can be activated either through a command-line flag (`--use_liger_kernel true`) or, in our case, by setting `LIGER_KERNEL=true` in the SLURM script.

By enabling Liger kernels, we observe a dramatic reduction in memory usage—nearly half—and a substantial increase in training throughput.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	**Training Throughput**	**Memory (GB)**
15.8	1	14	bf16	bf16	sdpa	true	**36.479**	**32,46**

Table 15.8 – Liger kernels reduce memory usage by ~50% and improve throughput by 32%.

Taking advantage of the extra memory provided by Liger kernels, we increase the batch size further. The training system now supports a batch size of 37—more than 6× the original baseline—while reaching a throughput of over 48,000 tokens/s per GPU.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	**Training Throughput**	**Memory (GB)**
15.9	1	37	bf16	bf16	sdpa	true	**48.081**	**62,33**

Table 15.9 – Final optimized configuration achieves 4.5× throughput improvement over the baseline.

When supported by the model architecture, enabling Liger produces a dramatic reduction in memory consumption—close to 50% in our experiments—together with a substantial increase in training throughput on the NVIDIA H100.

However, Liger is not currently supported for all Transformer architectures. In particular, when running with facebook/opt-1.3b, the Hugging Face runtime reports in the standard output:

```
INFO - There are currently no Liger kernels supported for
model type: opt.
```

As a consequence, no performance or memory improvement is observed for OPT when enabling Liger, while LLaMA 3.2 fully benefits from this optimization.

This architectural dependency makes Liger an excellent example of how low-level kernel optimizations are model-specific, and why validating backend support is essential in performance engineering.

Task 15.8 – Using the Liger kernel

Starting from the configuration obtained in Task 15.7 (full bf16 precision with Flash Attention), enable Liger kernels by setting `LIGER_KERNEL=true` in the SLURM script.

Proceed in two stages:

- First, run the experiment with LLaMA 3.2 (1B) and verify that your measured training throughput and memory usage are consistent with the reference values reported in Table 15.8.
- Then, repeat exactly the same experiment using facebook/opt-1.3b.

For both models:

- Inspect the standard output log and identify whether Liger is actually activated at runtime.
- Record training throughput and peak GPU memory usage.

Finally, compare and discuss the results obtained with LLaMA and OPT, explicitly addressing:

- Why LLaMA exhibits a strong performance and memory improvement.
- Why OPT shows no improvement, based on the runtime support message.
- What this teaches you about the model-dependence of kernel-level optimizations.

Final Remarks

Through the systematic application of increasingly advanced optimization techniques—batch size scaling, mixed precision, full model bf16 precision, Flash Attention, and finally Liger kernels—we have achieved a 4.5× increase in training throughput on a single NVIDIA H100 GPU, without modifying the Python model code.

Each optimization layer builds upon the previous one, illustrating how hardware-aware, kernel-level, and numerical-precision optimizations interact to unlock increasingly higher levels of performance. At the same time, the OPT experiments clearly demonstrate an essential practical lesson: not all advanced optimizations are universally supported across model families. Liger kernels currently benefit LLaMA architectures, but not OPT, reinforcing the importance of inspecting backend support and runtime logs during performance tuning.

Interestingly, while Liger kernels are incompatible with `torch.compile()` at the time of writing, this limitation opens an opportunity for OPT-based optimization. As a final reflective exercise, students are encouraged to explore whether `torch.compile()` can recover part of the performance gap for OPT, and to compare the nature of compiler-level versus kernel-level optimizations.

Task 15.9 – Final Optimizations

Final Batch Size Scaling with Liger (LLaMA)
Since Liger kernels are currently supported only for LLaMA-type models, the batch size scaling experiment with Liger is to be performed exclusively with LLaMA 3.2 (1B).
Using the optimized configuration obtained in Task 15.8 (bf16 + Flash Attention + Liger), progressively increase the batch size until an Out-Of-Memory (OOM) condition is reached.
For this final LLaMA configuration, determine and record:

- The maximum batch size enabled by Liger kernels
- The corresponding training throughput
- The resulting peak GPU memory usage

Then, compare these results with those obtained in Task 15.7 (Flash Attention without Liger) and analyze:

- How much additional scaling headroom is enabled by kernel-level optimization
- Whether compute or memory is now the dominant bottleneck
- How the combined use of bf16 + Flash Attention + Liger reshapes the single-GPU performance envelope

Compiler-Based Optimization (OPT)

Because Liger kernels are not supported for facebook/opt-1.3b, use this model to explore an alternative optimization path based on compilation.
Starting from the optimized OPT configuration of Task 15.7 (bf16 + Flash Attention, without Liger):

- Enable torch.compile
- Re-run the training experiment
- Measure training throughput and memory usage

Then, compare the results with those obtained without compilation, and reflect on the following:

- To what extent can compiler-level optimizations compensate for the absence of kernel-level optimizations such as Liger?
- How do the performance gains of `torch.compile()` compare conceptually to those obtained with Flash Attention and Liger?
- What does this tell you about the complementary roles of compilers and custom GPU kernels in modern LLM optimization pipelines?

By completing this chapter, students gain direct hands-on insight into how modern LLM training performance emerges from the careful orchestration of numerical precision, memory behavior, kernel implementations, and architectural support—skills that are directly transferable to multi-GPU and distributed training scenarios.

15.3 Scaling Up: Distributed Training Across Multiple GPUs

Training Large Language Models efficiently requires not only powerful accelerators but also software infrastructures capable of orchestrating computation across many GPUs and, potentially, across multiple nodes.

Advanced frameworks such as DeepSpeed, Megatron-LM, or NVIDIA NeMo offer sophisticated solutions for data, tensor, and pipeline parallelism. However, their complexity lies beyond the scope of this book, which focuses on building a solid and practical foundation for scalable training.

In this section, we scale the training of the same LLM optimized throughout this chapter on the MareNostrum 5 supercomputer using pure data parallelism through PyTorch's Distributed Data Parallel (DDP) backend, accessed transparently via the Hugging Face Trainer.

One of the key advantages of using the Trainer API is that it abstracts away most of the complexity of distributed execution. The very same training script used for single-GPU execution can be launched across multiple GPUs without any modification. Under the hood, Distributed Data Parallel replicates the model on each GPU, distributes mini-batches across devices, and synchronizes gradients during backpropagation using high-performance collective operations. This enables efficient data parallelism, provided that the per-GPU batch size is large enough to amortize communication overhead.

Experimental Setup

We reuse the fully optimized single-GPU configuration obtained in Task 15.9 as our baseline:

- Model precision: `bf16`
- Mixed precision: `bf16`
- Attention: `Flash Attention (sdpa)`
- Kernel: `Liger`
- Batch size: 37 per GPU

To scale this configuration, we simply modify the number of nodes and the `GPUS_PER_NODE` variable in the SLURM script. All other parameters remain unchanged. This guarantees that any performance variation observed is exclusively due to scaling effects, not to changes in model configuration or numerical setup.

Scaling Results

The scaling results are shown in Table 15.10.

Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	**Training Throughput**	**Memory (GB)**
1	37	bf16	bf16	sdpa	true	**48,229**	**62.34**
2	37	bf16	bf16	sdpa	true	**46,188**	**62.21**
4	37	bf16	bf16	sdpa	true	**43,432**	**61.97**
8	37	bf16	bf16	sdpa	true	**42,297**	**62.13**
16	37	bf16	bf16	sdpa	true	**41,869**	**61.99**
32	37	bf16	bf16	sdpa	true	**41,424**	**61.99**

Table 15.10 – Per-GPU training throughput and memory usage when scaling training of the model across multiple GPUs on MareNostrum 5.

From Table 15.10, we observe that the per-GPU training throughput remains remarkably stable as the number of GPUs increases. While it decreases from 48,229 tokens/s on a single GPU to 41,424 tokens/s at 32 GPUs, the degradation is gradual and well contained. This indicates that the optimized single-GPU pipeline—combining bf16 precision, Flash Attention, and Liger kernels—remains highly efficient even under distributed execution. The GPU memory usage stays essentially constant at around 62 GB across all configurations, confirming that data parallelism does not introduce additional memory pressure per device, as expected.

Table 15.11 complements this view by presenting the aggregated system-level performance. Total throughput scales from 48,229 tokens/s on one GPU to more than 1.32 million tokens/s on 32 GPUs, corresponding to a 27.48× speedup. This is very close to the theoretical ideal of 32×, resulting in a parallel efficiency of 86%, which is an excellent figure for distributed training at this scale.

Num GPUs	Per-GPU Throughput	Total Throughput	Speedup vs 1 GPU	Efficiency (%)
1	48,229	48,229	1.00×	100%
2	46,188	92,376	1.92×	96%
4	43,432	173,728	3.60×	90%
8	42,297	338,376	7.01×	88%
16	41,869	669,904	13.89×	87%
32	41,424	1,325,568	27.48×	86%

Table 15.11 – Total training throughput, relative speedup, and parallel efficiency of distributed training using up to 32 GPUs. Efficiency is computed as Speedup divided by the number of GPUs.

The progressive drop in efficiency—from 100% on one GPU to 96% on two, 90% on four, and stabilizing around 86–88% at higher GPU counts—reflects the growing impact of gradient synchronization and collective communication overheads. As the number of workers increases, the cost of all-reduce operations in Distributed Data Parallel becomes more significant, and shared network resources begin to saturate. Small additional overheads related to process coordination and kernel launches also accumulate.

Nevertheless, the fact that efficiency remains above 85% even at 32 GPUs demonstrates that the training pipeline is well balanced between computation and communication. This indicates that the per-GPU batch size of 37 is sufficiently large to amortize synchronization costs, and that the underlying interconnect and communication stack of MareNostrum 5 provide the necessary bandwidth and low latency required for efficient large-scale LLM training.

Overall, these results show that the optimization strategy developed throughout this chapter not only maximizes single-GPU performance, but also translates effectively into high distributed scalability. This confirms that careful attention to numerical precision, attention kernels, and low-level GPU optimizations is not only beneficial for standalone training, but is also a prerequisite for achieving high efficiency in multi-GPU and multi-node environments.

Task 15.10 – Scaling LLaMA on Multiple GPUs

Using the SLURM script from Task 15.9 as a starting point:

- Run the training experiment with 2, 4, 8, 16, and 32 GPUs.
- Adjust only `#SBATCH --nodes` and `GPUS_PER_NODE`
- Keep all other parameters unchanged.

For each configuration, measure and record:

- Per-GPU training throughput
- Total training throughput
- Peak GPU memory usage

Verify that your results are consistent with Tables 15.10 and 15.11 in terms of scaling trend, speedup, and efficiency.

Task 15.11 – Scaling OPT on Multiple GPUs

Repeat exactly the same scaling experiment using facebook/opt-1.3b, starting from the same optimized configuration (bf16 + Flash Attention, without Liger, as Liger is not supported for OPT, and compiled).

Again, measure:

- Per-GPU training throughput
- Total training throughput
- Peak GPU memory usage
- Relative speedup vs. the OPT 1-GPU optimized baseline
- Parallel efficiency

Task 15.12 – Analysis and comparison of LLaMA and OPT

Using the results obtained for both models, analyze and discuss:

- How close the observed speedup is to the theoretical ideal linear scaling in each case.
- Whether LLaMA exhibits better scalability than OPT, and why this may be happening.
- Whether communication overhead becomes the dominant bottleneck at large GPU counts for both models.

Summarize your conclusions in a comparative table and a short scaling analysis report if you consider it interesting.

15.4 Cost-Efficient Optimization: Performance per Euro

Does using more GPUs always pay off in practice? Training Large Language Models is not only a technical challenge—it is also an economic one. In both academic and industrial environments, computational resources are finite, and optimization must therefore address cost-efficiency, not just raw performance. In practical terms, this means understanding how many tokens per second are processed per euro spent.

When GPU usage is billed linearly with time—as is typical in cloud platforms and shared HPC systems such as MareNostrum 5—the following relations apply:

- Training cost is proportional to training time.
- Training time is proportional to the total number of tokens divided by total throughput.

As a result, minimizing training cost requires maximizing throughput per GPU-hour, not merely total throughput.

The scaling results obtained in the previous section reveal a critical trade-off: although total throughput increases almost linearly with the number of GPUs, parallel efficiency per GPU gradually decreases due to communication overheads and synchronization costs. This leads to a diminishing return in economic efficiency as more GPUs are added. Training finishes sooner in wall-clock time, but the cost per trained token increases.

Table 15.12 quantifies this effect by normalizing throughput with respect to GPU usage, yielding a proxy metric for cost efficiency (tokens/s/€). As shown in both the table and Figure 15.1, the cost efficiency monotonically decreases as the number of GPUs increases. While a single GPU provides the highest efficiency per euro, configurations with 4 or 8 GPUs often represent an effective sweet spot, balancing turnaround time, budget constraints, and energy consumption.

This analysis highlights a key principle in large-scale AI training: faster is not always cheaper. Under tight budget or sustainability constraints, selecting a moderately parallel configuration can offer the best compromise between time-to-solution and economic efficiency.

.

Num GPUs	Total Throughput	Efficiency (Tokens/s/GPU)	Cost Efficiency (Tokens/s/€)
1	48,229	100%	1.00×
2	92,376	96%	0.96×
4	173,728	90%	0.90×
8	338,376	88%	0.88×
16	669,904	87%	0.87×
32	1,325,568	86%	0.86×

Table 15.12 – Cost-efficiency analysis based on throughput per GPU and normalized economic cost per token processed. Assumes linear billing proportional to GPU time.

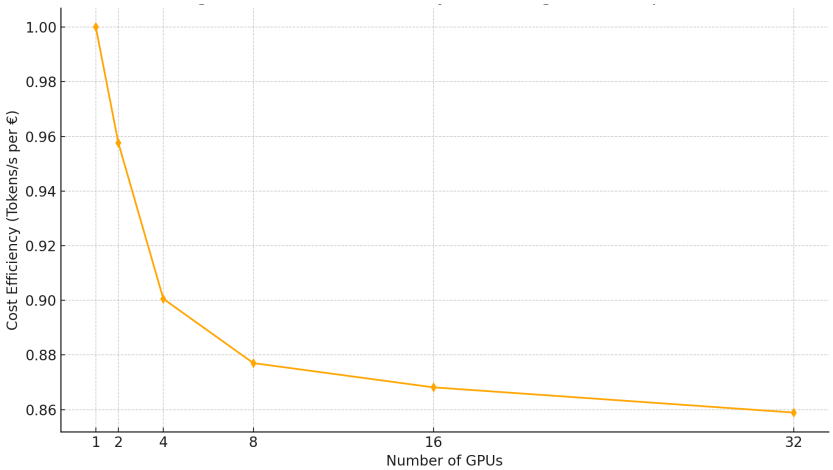

Figure 15.1 – Cost-efficiency of training (proxy metric: tokens/s/€). Although total throughput scales, the efficiency per euro slightly declines with more GPUs.

Task 15.13 – Cost-Efficiency Analysis for OPT

Using the distributed scaling results obtained for facebook/opt-1.3b in Task 15.11, build the cost-efficiency table equivalent to Table 15.12.

Proceed as follows:

- Use the measured total throughput values for 1, 2, 4, 8, 16, and 32 GPUs.
- Normalize all values with respect to the single-GPU case.
- Compute for each configuration: relative throughput, parallel efficiency, and relative cost efficiency (tokens/s/€)

Present your results in a table analogous to Table 15.12, and compare them with the LLaMA-based cost-efficiency analysis.

Finally, compare the economic scaling behavior of LLaMA and OPT and discuss:

- Which model exhibits better cost-efficiency at scale.
- Whether the optimal "sweet spot" in number of GPUs shifts between both models.

Task 15.14 – Final Reflections and Cost–Performance Analysis

Summarize and critically analyze your findings from this lab using the following guiding questions. Your discussion should be quantitative whenever possible, supported by the measurements obtained throughout the chapter for both LLaMA 3.2 and facebook/opt-1.3b.

- What throughput improvement did you achieve in Task 15.9 compared to the original baseline in Task 15.1? Express the improvement as a multiplicative factor.
- Which optimization stage produced the largest single performance jump (mixed precision, full bf16 model precision, Flash Attention, or Liger)? Why do you think this optimization was so impactful at the hardware level?
- If the budget is strictly limited, which configuration offers the best performance per euro, and why?
- Considering performance, cost, and energy efficiency together, which configuration would you choose in a real-world production scenario, and why?
- Conclude with your perspective on the importance of performance tuning in LLM training workflows, and reflect on how the insights gained in this lab can be applied to other Transformer-based models.

This chapter has demonstrated the value of progressive, layered optimization—starting from a single GPU and extending to distributed environments. Through careful tuning of model precision, attention kernels, and distributed training settings, we achieved:

- 4.5× improvement in throughput on a single GPU via software-level optimizations.
- 27.5× speedup when scaling across 32 GPUs with minimal code changes.
- High parallel efficiency (≥86%) thanks to proper batch sizing and use of communication-efficient kernels.
- A clear view of cost-efficiency trade-offs, reinforcing the importance of balancing speed with resource constraints.

In large-scale HPC environments, these optimizations translate into faster turnaround**,** lower energy use, and reduced cost per experiment—making AI research more accessible and sustainable.

Importantly, these performance gains were achieved without modifying the training script itself, thanks to the robustness of PyTorch DDP and Hugging Face's `Trainer` class—underscoring how the right software stack simplifies scalable LLM training on HPC systems.

Part V brought together all the foundational elements introduced throughout the book—software environments, hardware-aware optimization, distributed training—to tackle the complexity of training state-of-the-art LLMs. By combining empirical benchmarks with conceptual clarity, we hope this final section enables readers to navigate real-world challenges in AI-scale supercomputing.

15.5 Key Takeaways from Chapter 15

> Training efficiency is just as important as achieving high model performance when working with Large Language Models (LLMs). This chapter focused on extracting maximum performance per GPU and scaling effectively across many GPUs using real-world tools and datasets.

> We began with a baseline configuration and progressively applied software-level optimizations.

> Scaling the batch size allowed us to fully utilize available GPU memory.

> Enabling mixed precision helped reduce memory usage and improve throughput.

> Switching to full model precision using bf16 further increased performance while maintaining numerical stability.

> Replacing standard attention with Flash Attention (sdpa) improved both throughput and memory footprint.

> Enabling Liger kernels delivered the largest gains, resulting in a 4.5× increase in training throughput compared to the baseline and allowing a 6× larger batch size on a single GPU.

> The final single-GPU configuration reached 48,081 tokens/s, compared to 10,485 tokens/s at baseline—demonstrating how fine-grained optimizations can dramatically accelerate training even without changing the model architecture.

> We then scaled the best configuration across 2, 4, 8, 16, and 32 GPUs on the MareNostrum 5 supercomputer.

> Total throughput scaled almost linearly, but per-GPU efficiency declined from 100% (1 GPU) to 86% (32 GPUs), revealing the cost of inter-GPU communication and system overheads.

> There exists a trade-off between speed and cost-efficiency: although 32 GPUs provide the fastest training, the most cost-effective configuration may lie at 4 or 8 GPUs depending on the budget and time constraints.

> Hugging Face's Trainer API simplified the deployment of experiments from 1 to 32 GPUs with minimal code changes, showing the value of combining high-level tools with high performance systems.

> The chapter demonstrated the importance of layered optimization: students should first optimize training on a single GPU before attempting distributed scaling.